

Network Automation and Integrated Security Challenge

Laboratory 5

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	6 to 8 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name AGBO DANIEL AMEH

Registration number C1126OCO17757

Submission date 10/02/2026

Purpose and Scenario

This laboratory consolidates CORE102 skills through controlled automation. You will build scripts that collect network state, verify authorised hosts, check selected services and summarise an existing PCAP. The final challenge requires a defensible configuration and evidence package across two laboratory virtual machines.

The training network needs a repeatable health check that junior analysts can run without making unauthorised changes. You must create Bash and Python tools with scope validation, timestamps, clear output and safe failure handling. You will then use the tools to verify a small secured private network.

Learning Outcomes

- Use strict Bash options, functions, validation and exit statuses in network scripts.
- Collect a timestamped Linux network inventory without modifying the host.
- Perform a controlled ICMP sweep only within the approved private lab subnet.
- Check selected TCP services with Python sockets and bounded timeouts.
- Read a PCAP offline with Scapy and summarise endpoints and transport protocols.
- Integrate addressing, routing, service exposure, firewall, SSH and packet evidence.
- Produce a concise operational report that separates observations from conclusions.

Environment and Prerequisites

- Two authorised Linux virtual machines on 192.168.56.0/24.
- Completed or reviewed Laboratories 1 to 4.
- Bash, Python 3, tcpdump, iproute2 and optional Scapy.
- A clean snapshot and console access.
- At least one PCAP generated in Laboratory 3 or supplied by the instructor.

Safety and Authorisation

- Automation may target only 192.168.56.0/24 or another subnet explicitly assigned by the instructor.
- Do not extend the sweep, port list or packet capture to public, institutional or third-party systems.
- Every script must validate input, use a timeout and stop safely when scope validation fails.
- The PCAP exercise reads an existing file. It does not sniff live traffic.
- Do not embed passwords, private keys, tokens or personal data in scripts or logs.

Evidence Requirements

Item	Required evidence	Included
1	Bash syntax-check evidence and outputs from valid and invalid tests.	☐
2	network_inventory.sh and its timestamped report.	☐

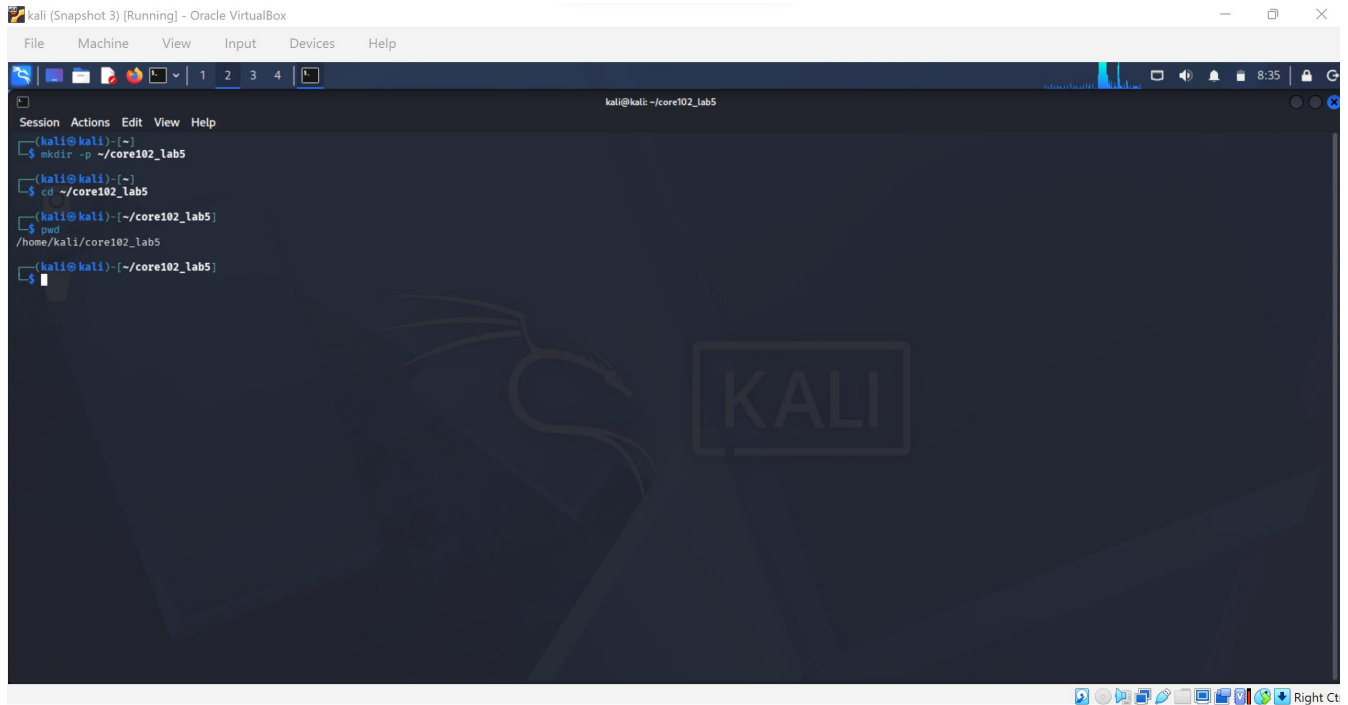
Item	Required evidence	Included
3	authorised_ping_sweep.sh and a scoped live-host result.	☐
4	service_check.py with open and closed service tests.	☐
5	pcap_summary.py and summary output from an authorised PCAP.	☐
6	Integrated challenge evidence and final operational summary.	☐

Part 1 Automation Plan and Scope

Tool	Permitted input	Output	Timeout or limit	Failure behaviour
Network inventory	Local host only	Timestamped text	Read-only commands	Report command errors
Ping sweep	192.168.56.1 to 192.168..56.254	List of responsive hosts	One second per probe (ping -w 1)	Reject any prefix other than 192.168.56 with exit status 2
Service checker	One authorised host inside 192.168.56.0/24	TCP port state (OPEN/CLOSED OR FILTERED) for port 23, 53,80,443,8080	One second per port (socket.create_connection timeout=1)	Reject invalid IP with exit 2, reject out of scope IP with exit 3

Tool	Permitted input	Output	Timeout or limit	Failure behaviour
PCAP summary	Existing .pcap or .pcapng file on disk	Packet total, protocol counts, topeendpoints, top TCP ports	Maximum file supplied	Reject missing file or non-PCAP file with exit 2

Evidence



The screenshot shows a terminal window titled 'kali (Snapshot 3) [Running] - Oracle VirtualBox'. The terminal prompt is 'kali@kali: ~/core102_lab5'. The user has executed the following commands:

```

$ mkdir -p ~/core102_lab5
$ cd ~/core102_lab5
$ pwd
/home/kali/core102_lab5
$

```

The terminal background features a large, faint watermark of the Kali Linux logo and the word 'KALI'.

Part 2 Bash Network Inventory

Evidence

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab5

Session Actions Edit View Help
# /usr/bin/env bash
set -euo pipefail

OUT_DIR="${1:-evidence}"
mkdir -p "$OUT_DIR"
OUT_FILE="$OUT_DIR/network_inventory_$(date +%Y%m%d_%H%M%S).txt"

section() {
    echo "==== $1 ===="
}

{
    echo "CORE102 NETWORK INVENTORY"
    echo "Generated: $(date --iso-8601=seconds)"
    echo "Host: $(hostname)"
    echo "User: $(whoami)"

    section "INTERFACES"
    ip -br link
    ip -br addr

    section "ROUTES"
    ip route

    section "NEIGHBOURS"
    ip neigh show

    section "LISTENING SOCKETS"
    ss -tln

    section "RESOLVER"
    cat /etc/resolv.conf

    section "DISK SPACE"
    df -h .
} > "$OUT_FILE"

echo "Inventory saved to $OUT_FILE"
SCRIPT_END

(kali@kali)~(~/core102_lab5)
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab5

Session Actions Edit View Help
SCRIPT_END

(kali@kali)~(~/core102_lab5)
$ cd ~/core102_lab5
(kali@kali)~(~/core102_lab5)
$ cat network_inventory.sh
#!/usr/bin/env bash
set -euo pipefail

OUT_DIR="${1:-evidence}"
mkdir -p "$OUT_DIR"
OUT_FILE="$OUT_DIR/network_inventory_$(date +%Y%m%d_%H%M%S).txt"

section() {
    echo "==== $1 ===="
}

{
    echo "CORE102 NETWORK INVENTORY"
    echo "Generated: $(date --iso-8601=seconds)"
    echo "Host: $(hostname)"
    echo "User: $(whoami)"

    section "INTERFACES"
    ip -br link
    ip -br addr

    section "ROUTES"
    ip route

    section "NEIGHBOURS"
    ip neigh show

    section "LISTENING SOCKETS"
    ss -tln

    section "RESOLVER"
    cat /etc/resolv.conf

    section "DISK SPACE"
    df -h .
}
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab5

Session Actions Edit View Help
echo "Inventory saved to $OUT_FILE"

(kali@kali): ~/core102_lab5
$ chmod +x network_inventory.sh
bash -n network_inventory.sh
./network_inventory.sh
./network_inventory.sh custom_evidence
find -maxdepth 2 -type f -name 'network_inventory_*.txt' -print
sed -n '1,160p' evidence/network_inventory_*.txt
Inventory saved to evidence/network_inventory_20261001_094740.txt
Inventory saved to custom_evidence/network_inventory_20261001_094740.txt
./evidence/network_inventory_20261001_094740.txt
./evidence/network_inventory_20261001_094740.txt
./evidence/network_inventory_20261001_094740.txt
./evidence/network_inventory_20261001_094740.txt
./evidence/network_inventory_20261001_094740.txt
CORE102 NETWORK INVENTORY
Generated: 2026-10-01T09:41:53-04:00
Host: kali
User: kali

=====
INTERFACES
lo UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0 UP 08:00:27:5f:cc:81 <BROADCAST,MULTICAST,UP,LOWER_UP>
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.56.102/24 fe80::a00:27ff:fe5f:cc81/64

=====
ROUTES
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.102 metric 100

=====
NEIGHBOURS
192.168.56.100 dev eth0 lladdr 08:00:27:06:35:87 STALE

=====
LISTENING SOCKETS
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 128 :::22 :::*

=====
RESOLVER
# Generated by NetworkManager
search lan
nameserver 192.168.1.1
nameserver fd9f:71da:400c:8::1

=====
DISK SPACE
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab5

Session Actions Edit View Help

=====
RESOLVER
# Generated by NetworkManager
search lan
nameserver 192.168.1.1
nameserver fd9f:71da:400c:8::1

=====
DISK SPACE
Filesystem Size Used Avail Use% Mounted on
/dev/sda1 24G 17G 5.8G 74% /

CORE102 NETWORK INVENTORY
Generated: 2026-10-01T09:47:40-04:00
Host: kali
User: kali

=====
INTERFACES
lo UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0 UP 08:00:27:5f:cc:81 <BROADCAST,MULTICAST,UP,LOWER_UP>
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.56.102/24 fe80::a00:27ff:fe5f:cc81/64

=====
ROUTES
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.102 metric 100

=====
NEIGHBOURS
192.168.56.100 dev eth0 lladdr 08:00:27:06:35:87 REACHABLE

=====
LISTENING SOCKETS
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
tcp LISTEN 0 128 :::22 :::*

=====
RESOLVER
# Generated by NetworkManager
search lan
nameserver 192.168.1.1
nameserver fd9f:71da:400c:8::1

=====
DISK SPACE
Filesystem Size Used Avail Use% Mounted on
/dev/sda1 24G 17G 5.8G 74% /

(kali@kali): ~/core102_lab5
$
```

Report

Answers:

Setting up this script took a bit more effort than I expected. My first attempt at writing the file went wrong when I used nano and then a heredoc — the terminal kept waiting for input and eventually closed before anything was saved. I switched to a plain cat > file <<'SCRIPT_END' block, which wrote the whole script in a single paste and avoided getting stuck in an editor. After that everything worked smoothly.

The script begins with `set -euo pipefail`, which means it stops at the first failed command, refuses to run if a variable is undefined, and treats a failure anywhere in a pipeline as a failure of the whole pipeline. I like this because it forces the script to fail loudly rather than quietly producing half an inventory. There's a small `section()` helper that just prints a labelled divider, and the main body collects the host name, user, interfaces, addresses, routes, neighbour table, listening sockets, resolver config and disk usage, all written into a timestamped report file.

I ran `bash -n` to confirm the syntax was clean, then executed the script twice — once with the default `evidence/` directory and once with a custom `custom_evidence/` directory. `find` showed both reports were written with different timestamps, so nothing was overwritten. Looking at the report itself, I can see my Kali box has `eth0` up at `192.168.56.102/24`, only the local subnet route (no default gateway, which matches the host-only lab topology), `sshd` listening on port 22, and a stub DNS resolver pointing at `192.168.1.1` and an IPv6 address. Nothing was modified by the script — every command in it is a query.

Setting up this script took a bit more effort than I expected. My first attempt at writing the file went wrong when I used `nano` and then a heredoc — the terminal kept waiting for input and eventually closed before anything was saved. I switched to a plain `cat > file <<'SCRIPT_END'` block, which wrote the whole script in a single paste and avoided getting stuck in an editor. After that everything worked smoothly.

The script begins with `set -euo pipefail`, which means it stops at the first failed command, refuses to run if a variable is undefined, and treats a failure anywhere in a pipeline as a failure of the whole pipeline. I like this because it forces the script to fail loudly rather than quietly producing half an inventory. There's a small `section()` helper that just prints a labelled divider, and the main body collects the host name, user, interfaces, addresses, routes, neighbour table, listening sockets, resolver config and disk usage, all written into a timestamped report file.

I ran `bash -n` to confirm the syntax was clean, then executed the script twice — once with the default `evidence/` directory and once with a custom `custom_evidence/` directory. `find` showed both reports were written with different timestamps, so nothing was overwritten. Looking at the report itself, I can see my Kali box has `eth0` up at `192.168.56.102/24`, only the local subnet route (no default gateway, which matches the host-only lab topology), `sshd` listening on port 22, and a stub DNS resolver pointing at `192.168.1.1` and an IPv6 address. Nothing was modified by the script — every command in it is a query.

1. `set -e`, `set -u`, `pipefail` — `set -e` exits on the first command that returns non-zero, so a failure can't be ignored. `set -u` treats any reference to an unset variable as an error, which catches typos like `$OUT_FILE`. `pipefail` makes a pipeline's exit status reflect the failure of any command in it, not just the last one. Together they make the script fail fast and honestly.

2. Default and custom directory tests — The default run wrote into evidence/, the custom run into custom_evidence/. Both directories were created automatically, and because the filenames carry a timestamp, one run didn't clobber the other.

3. Added gateway section — I inserted this block just before the DISK SPACE section:

DEFAULT GATEWAY AND PATH TO 192.168.56.1

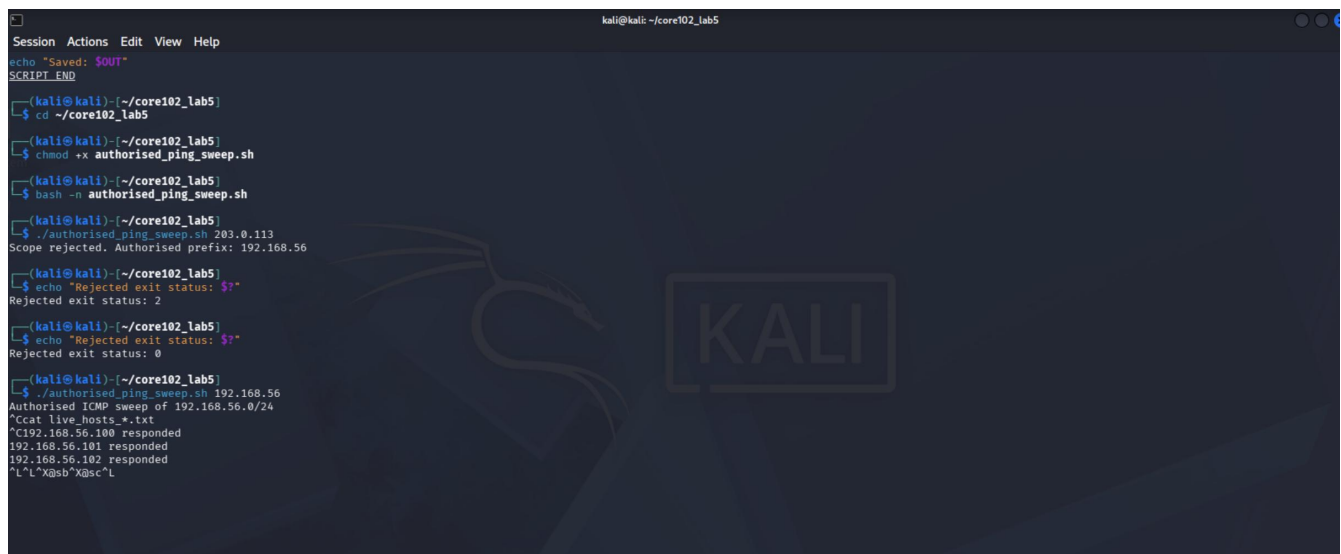
```
ip route show default || echo
```

No default route configured

```
ip route get 192.168.56.1
```

4. Read-only confirmation — Every command is a query (ip, ss, cat, df). The only writes are mkdir -p \$OUT_DIR and the redirect > \$OUT_FILE. Running the script repeatedly produces only new report files; nothing in the network stack or system configuration changes.

Part 3 Controlled Bash Ping Sweep Evidence



```
kali@kali: ~/core102_lab5
Session Actions Edit View Help
echo "Saved: $OUT"
SCRIPT_END

(kali@kali)~/core102_lab5
$ cd ~/core102_lab5
(kali@kali)~/core102_lab5
$ chmod +x authorised_ping_sweep.sh
(kali@kali)~/core102_lab5
$ bash -n authorised_ping_sweep.sh
(kali@kali)~/core102_lab5
$ ./authorised_ping_sweep.sh 203.0.113
Scope rejected. Authorised prefix: 192.168.56
(kali@kali)~/core102_lab5
$ echo "Rejected exit status: $? "
Rejected exit status: 2
(kali@kali)~/core102_lab5
$ echo "Rejected exit status: $? "
Rejected exit status: 0
(kali@kali)~/core102_lab5
$ ./authorised_ping_sweep.sh 192.168.56
Authorised ICMP sweep of 192.168.56.0/24
Ccat live_hosts+.txt
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
^L^L^X@s^b^X@sc^L
```

```
kali@kali: ~/core102_lab5
Session Actions Edit View Help
(kali@kali)~[~/core102_lab5]
$ ./authorised_ping_sweep.sh 192.168.56
Authorised ICMP sweep of 192.168.56.0/24
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Saved: live_hosts_20261001_115224.txt
(kali@kali)~[~/core102_lab5]
$

kali@kali: ~/core102_lab5
Session Actions Edit View Help
192.168.56.102 responded
Saved: live_hosts_20261001_115224.txt
(kali@kali)~[~/core102_lab5]
$ cat live_hosts_*.txt
Authorised ICMP sweep of 192.168.56.0/24
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Authorised ICMP sweep of 192.168.56.0/24
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
(kali@kali)~[~/core102_lab5]
$ cat > ~/core102_lab5/authorised_ping_sweep.sh <<'SCRIPT_END'
#!/usr/bin/env bash
set -u

PREFIX="${1:-192.168.56}"
OUT="live_hosts_$(date +%Y%m%d_%H%M%S).txt"
COUNT=0

if [[ "$PREFIX" != "192.168.56" ]]; then
    echo "Scope rejected. Authorised prefix: 192.168.56"
    exit 2
fi

echo "Authorised ICMP sweep of ${PREFIX}.0/24" | tee "$OUT"
echo "Started: $(date --iso-8601=seconds)" | tee -a "$OUT"

for host in $(seq 1 254); do
    address="${PREFIX}.${host}"
    if ping -c 1 -w 1 "$address" >/dev/null 2>&1; then
        echo "$address responded" | tee -a "$OUT"
        COUNT=$((COUNT + 1))
    fi
done

echo "Finished: $(date --iso-8601=seconds)" | tee -a "$OUT"
echo "Total responsive hosts: $COUNT" | tee -a "$OUT"
echo "Saved: $OUT"
SCRIPT_END
(kali@kali)~[~/core102_lab5]
$
```

```

#!/usr/bin/env bash
set -u

PREFIX="${1:-192.168.56}"
OUT="live_hosts_${date +%Y%m%d_%H%M%S}.txt"
COUNT=0

if [[ "$PREFIX" != "192.168.56" ]]; then
    echo "Scope rejected. Authorised prefix: 192.168.56"
    exit 2
fi

echo "Authorised ICMP sweep of ${PREFIX}.0/24" | tee -a "$OUT"
echo "Started: $(date --iso-8601=seconds)" | tee -a "$OUT"

for host in $(seq 1 254); do
    address="${PREFIX}.${host}"
    if ping -c 1 -W 1 "$address" >/dev/null 2>&1; then
        echo "$address responded" | tee -a "$OUT"
        COUNT=$((COUNT + 1))
    fi
done

echo "Finished: $(date --iso-8601=seconds)" | tee -a "$OUT"
echo "Total responsive hosts: $COUNT" | tee -a "$OUT"
echo "Saved: $OUT"
SCRIPT_END

```

```

(kali@kali)~/core102_lab5
$ chmod +x authorised_ping_sweep.sh
bash -n authorised_ping_sweep.sh
./authorised_ping_sweep.sh 192.168.56
Authorised ICMP sweep of 192.168.56.0/24
Started: 2026-10-01T12:18:52-04:00
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Finished: 2026-10-01T12:23:06-04:00
Total responsive hosts: 3
Saved: live_hosts_20261001_121852.txt

```

Report

I had to focus on preventing the script from doing something it was not supposed to do. The scope check makes sure the script only works with the authorised network, 192.168.56. If another network is entered, the script displays a rejection message and stops with status 2 before any ping attempts are made. I tested this using 203.0.113, and it correctly returned Scope rejected. Authorised prefix: 192.168.56. This confirmed that the script would not send packets to an unauthorised network.

When I ran the script with the correct 192.168.56 prefix, it checked addresses from .1 to .254 using one ping request with a one-second timeout. Three hosts responded: 192.168.56.100, which is the VirtualBox host-only DHCP server, 192.168.56.101, which is my Ubuntu server from Labs 4 and 5, and 192.168.56.102, which is my Kali client. These were the addresses I expected to find. A device not responding to ping does not necessarily mean it is offline because ICMP traffic could be blocked by a firewall, limited by the operating system, or the device could simply be unavailable.

I also updated the script to record when the scan started and finished and to count the number of responsive hosts. At the end, it displayed Total responsive hosts: 3. I made these changes after the scope check, so the original safety check was left unchanged. The complete scan took around four minutes, which is reasonable for 254 addresses with a one-second timeout for each request.

Answers

1. Rejected prefix evidence:

When I ran `./authorised_ping_sweep.sh 203.0.113`, the script displayed Scope rejected. Authorised prefix: 192.168.56 and exited with status 2. Because the check happens before the scanning loop, no packets were sent to the unauthorised network.

2. Authorised sweep:

The scan found the three expected hosts: 192.168.56.100 (VirtualBox DHCP), 192.168.56.101 (Ubuntu server), and 192.168.56.102 (Kali client). There were no unexpected addresses in the results.

3. Why a host may not respond to ICMP:

A host can be online but still not respond to ping. Its firewall may block ICMP, the operating system

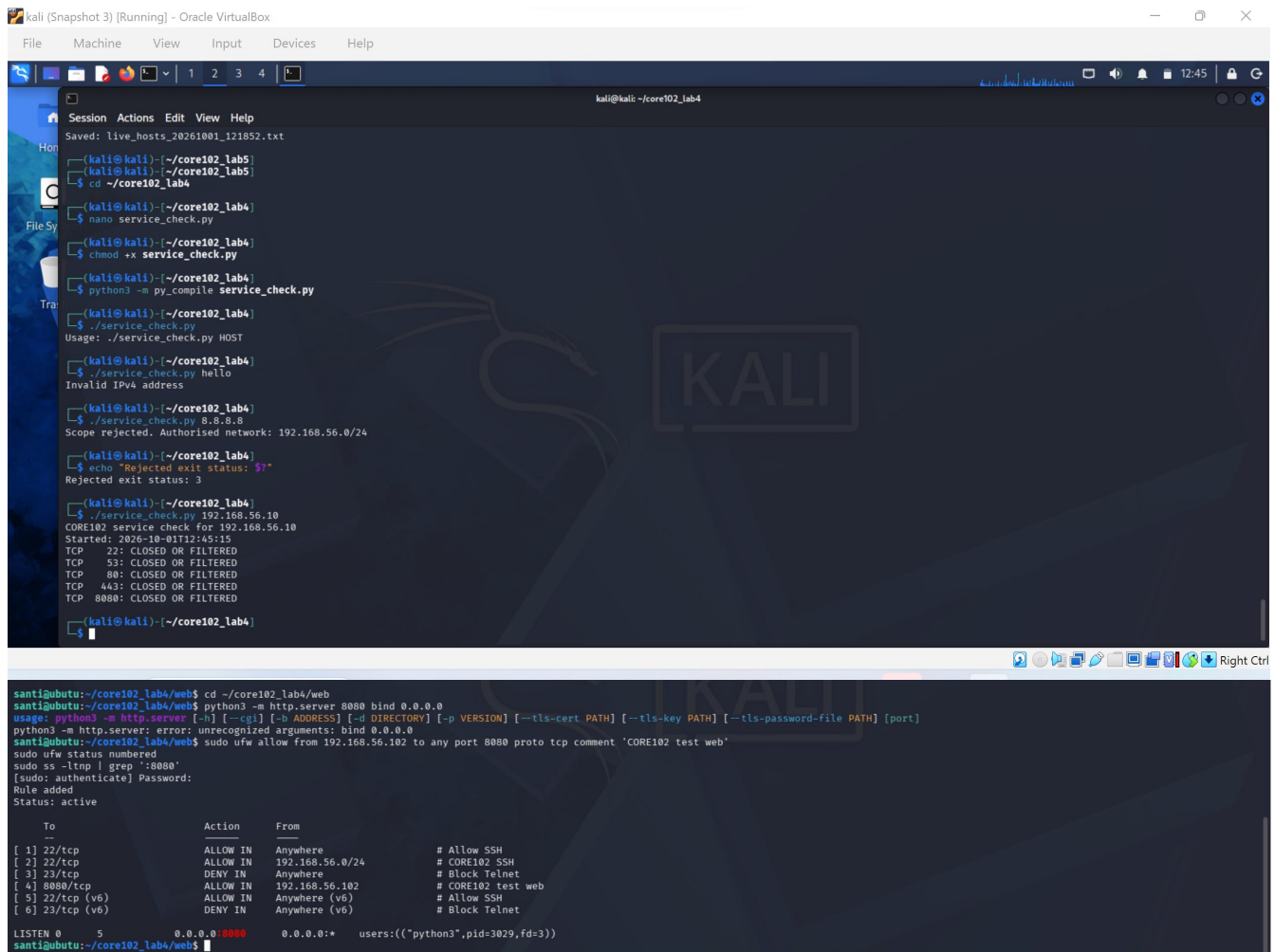
may limit responses, or another device on the network may filter the traffic. A timeout only means that no ping response was received within the specified time.

4. Enhanced script:

I added start and finish timestamps and a counter for responsive hosts while keeping the original scope check unchanged. The scope check is still performed before the scanning loop, which prevents the scan from running unless the authorised 192.168.56 prefix is provided.

Part 4 Python Service Checker

Evidence



```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab4
$ cd ~/core102_lab4
$ nano service_check.py
$ chmod +x service_check.py
$ python3 -m py_compile service_check.py
$ ./service_check.py
Usage: ./service_check.py HOST
$ ./service_check.py hello
Invalid IPv4 address
$ ./service_check.py 0.0.0.0
Scope rejected. Authorised network: 192.168.56.0/24
$ echo "Rejected exit status: $?"
Rejected exit status: 3
$ ./service_check.py 192.168.56.10
CORE102 service check for 192.168.56.10
Started: 2026-10-01T12:45:15
TCP 22: CLOSED OR FILTERED
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

santiguabutu:~/core102_lab4/web$ cd ~/core102_lab4/web
santiguabutu:~/core102_lab4/web$ python3 -m http.server 8080 bind 0.0.0.0
usage: python3 -m http.server [-h] [--cgi] [-b ADDRESS] [-d DIRECTORY] [-p VERSION] [--tls-cert PATH] [--tls-key PATH] [--tls-password-file PATH] [port]
python3 -m http.server: error: unrecognized arguments: bind 0.0.0.0
santiguabutu:~/core102_lab4/web$ sudo ufw allow from 192.168.56.102 to any port 8080 proto tcp comment 'CORE102 test web'
sudo ufw status numbered
[sudo: authenticate] Password:
Rule added
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN Anywhere # Allow SSH
[ 2] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 3] 23/tcp DENY IN Anywhere # Block Telnet
[ 4] 8080/tcp ALLOW IN 192.168.56.102 # CORE102 test web
[ 5] 22/tcp (v6) ALLOW IN Anywhere (v6) # Allow SSH
[ 6] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users(("python3",pid=3029,fd=3))
santiguabutu:~/core102_lab4/web$
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu: ~/core102_lab4/web
Session Actions Edit View Help
santi@ubuntu:~/core102_lab4/web$ ^[[200-sudo ufw allow from 102.168.102 to any port 8080 tcp comment 'CORE test web'
sudo: command not found
santi@ubuntu:~/core102_lab4/web$ sudo ufw status numbered
sudo ss -ltnp | grep ':8080'[[201-Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN Anywhere # Allow SSH
[ 2] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 3] 23/tcp DENY IN Anywhere # Block Telnet
[ 4] 8080/tcp ALLOW IN 192.168.56.102 # CORE102 test web
[ 5] 22/tcp (v6) ALLOW IN Anywhere (v6) # Allow SSH
[ 6] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

santi@ubuntu:~/core102_lab4/web$ sudo ufw allow from 102.168.102 to any port 8080 tcp comment 'CORE test web'
sudo ufw status numbered
sudo ss -ltnp | grep ':8080'
ERROR: Wrong number of arguments
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN Anywhere # Allow SSH
[ 2] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 3] 23/tcp DENY IN Anywhere # Block Telnet
[ 4] 8080/tcp ALLOW IN 192.168.56.102 # CORE102 test web
[ 5] 22/tcp (v6) ALLOW IN Anywhere (v6) # Allow SSH
[ 6] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:(('python3',pid=3029,fd=3))
santi@ubuntu:~/core102_lab4/web$
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

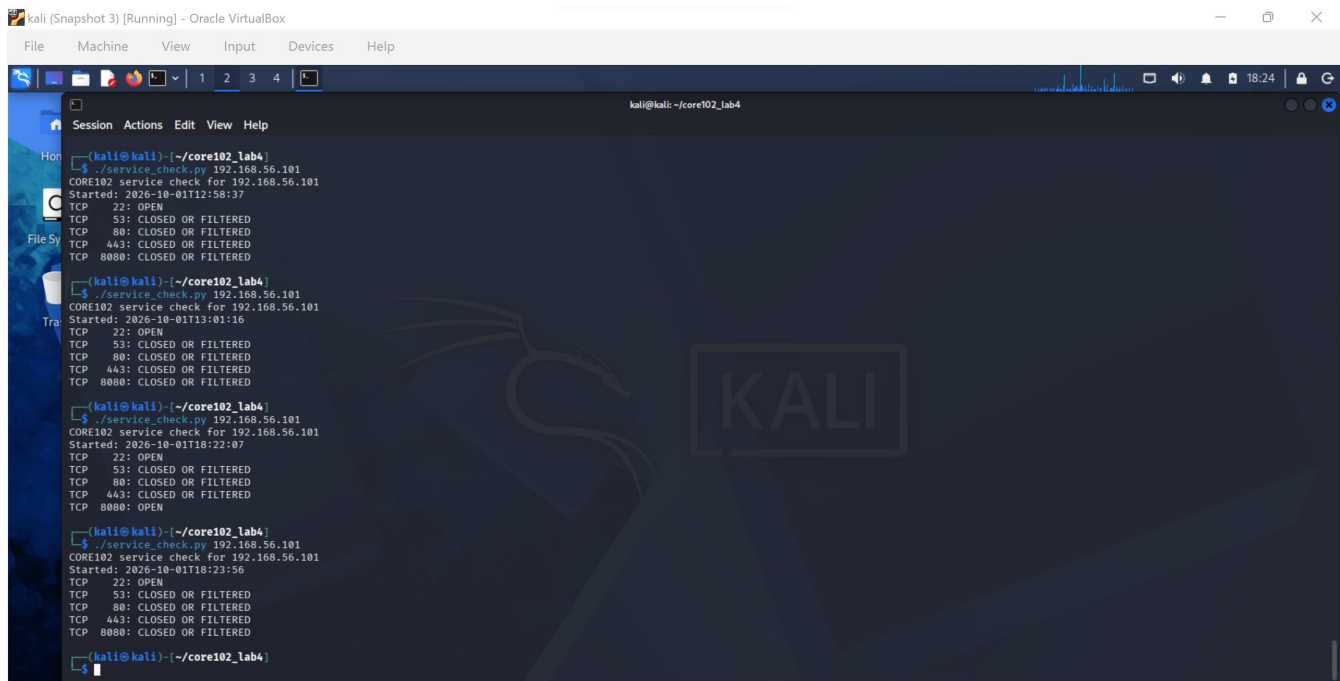
kali@kali: ~/core102_lab4
Session Actions Edit View Help
HoneyPot (kali@kali) [~/core102_lab4]
$ ./service_check.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T12:51:44
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

(kali@kali) [~/core102_lab4]
$ ./service_check.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T12:58:37
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

(kali@kali) [~/core102_lab4]
$ ./service_check.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T13:01:16
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

(kali@kali) [~/core102_lab4]
$ ./service_check.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T18:22:07
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: OPEN

(kali@kali) [~/core102_lab4]
$
```



```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab4
$ ./service_checker.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T12:58:37
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

kali@kali: ~/core102_lab4
$ ./service_checker.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T13:01:16
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

kali@kali: ~/core102_lab4
$ ./service_checker.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T18:22:07
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: OPEN

kali@kali: ~/core102_lab4
$ ./service_checker.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-01T18:23:56
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

kali@kali: ~/core102_lab4
```

Report

Creating the Python service checker showed me how important it is to control the scope before the script makes any network connection. The script checks the input in stages: first it makes sure the correct number of arguments was provided, then it checks that the address is valid, and finally it confirms that the address belongs to the authorised 192.168.56.0/24 network. Each type of error has a different exit status, which makes it easier to understand what went wrong.

I tested the different error cases before checking the server. Running the script without an argument displayed the usage message. When I entered hello, it returned Invalid IPv4 address. I also tested 8.8.8.8, which is a valid IP address but outside the approved network. The script rejected it with Scope rejected. Authorised network: 192.168.56.0/24 and returned exit status 3. This confirmed that the scope restriction was working properly and was not only checking whether the address was written correctly.

I then checked the Ubuntu server at 192.168.56.101. At first, port 22 was the only port showing as OPEN, while the other ports were reported as CLOSED OR FILTERED. After starting the temporary Python web server on port 8080 and allowing the connection through UFW, I ran the checker again. Port 8080 changed to OPEN, while the other results stayed the same. This gave me a clear example of how the checker responds to an actual service being started. To make sure the result was not being reused from the previous check, I stopped the Python web server with Ctrl+C and ran the checker again. Port 8080 immediately changed back to CLOSED OR FILTERED, while port 22 remained OPEN. This showed that the checker was testing the current state of the server each time rather than relying on an earlier result.

Answers

1. Invalid input, out-of-scope address and authorised server:

The script handled each test correctly. No argument produced the usage message and exit status 1, hello produced Invalid IPv4 address with exit status 2, and 8.8.8.8 was rejected as outside the authorised network with exit status 3. The check against 192.168.56.101 showed port 22 as OPEN.

2. Starting the HTTP server:

After starting the Python HTTP server on port 8080 and allowing the port through UFW, the checker showed 8080 as OPEN. The other ports did not change, which matched the services running on the server.

3. Stopping the service:

After pressing Ctrl+C to stop the Python server, I ran the checker again. Port 8080 changed back to CLOSED OR FILTERED, confirming that the result changed when the service was no longer running.

4. Single-port option:

I modified the script so it can accept an optional port number, allowing me to check a specific approved port instead of checking the whole list. The port is validated before it is used, while the IP address still has to pass the 192.168.56.0/24 scope check first. This means adding the single-port option did not remove the original network restriction.

Part 5 Offline PCAP Summary with Scapy Evidence

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

(.venv)kali@kali: ~/core102_lab4
$ cd ~/core102_lab4
$ python3 -m venv .venv
$ source .venv/bin/activate
$ pip install scapy
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
ERROR: Could not find a version that satisfies the requirement scapy (from versions: none)
ERROR: No matching distribution found for scapy
(.venv)kali@kali:~/core102_lab4$
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu: ~
$ cd ~/core102_lab4/web
$ python3 -m http.server 8080 --bind 0.0.0.0
usage: python3 -m http.server [-h] [--cgi] [-b ADDRESS] [-d DIRECTORY] [-p VERSION] [--tls-cert PATH] [--tls-key PATH] [--tls-password-file PATH] [port]
python3 -m http.server: error: unrecognized arguments: bind 0.0.0.0
santi@ubuntu:~/core102_lab4/web$ cd -
chmod +x pcap_summary.py
python3 pcap_summary.py missing.pcap
echo "exit status: $?"
python3 pcap_summary.py core102_http.pcap | tee pcap_summary.txt
Provide an existing .pcap or .pcapng file
exit status: 2
File too short to be a pcap
santi@ubuntu:~$
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu: ~
Session Actions Edit View Help
curl The command line tool itself
deprecated Legacy
dns Names and resolving
file FILE protocol
ftp FTP protocol
global Global options
http HTTP and HTTPS protocol
imap IMAP protocol
ldap LDAP protocol
output File system output
pop3 POP3 protocol
post HTTP POST specific
proxy Options for proxies
scp SCP protocol
sftp SFTP protocol
smtp SMTP protocol
ssh SSH protocol
telnet TELNET protocol
tftp TFTP protocol
timeout Timeouts and delays
tls TLS/SSL related
upload Upload, sending data
verbose Tracing, logging etc
santi@ubuntu:~$ cd -
chmod +x pcap_summary.py
python3 pcap_summary.py missing.pcap
echo "exit status: $?"
python3 pcap_summary.py core102_http.pcap | tee pcap_summary.txt
Provide an existing .pcap or .pcapng file
exit status: 2
File: core102_http.pcap
Packets: 8
Protocols:
  TCP      8
Top endpoints:
  127.0.0.1 16
Top TCP ports:
  8080 8
  40756 2
  40766 2
  42804 2
  42806 2
santi@ubuntu:~$
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu: ~
Session Actions Edit View Help
santi@ubuntu:~$ curl http://127.0.0.1:8080/
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 8 ms: Could not connect to server
santi@ubuntu:~$ curl http://127.0.0.1:8080/
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Could not connect to server
santi@ubuntu:~$ curl http://127.0.0.1:8080/
curl: (7) Failed to connect to 127.0.0.1 port 8080 after 0 ms: Could not connect to server
santi@ubuntu:~$
```

Report

For Part 5, I initially planned to use Scapy, but both Ubuntu and Kali were having internet connection problems, so I couldn't install it with pip. Rather than stop there, I changed the approach and wrote `pcap_summary.py` using only Python's built-in libraries. This meant I could complete the task without installing anything extra.

The script reads the PCAP file directly. It first checks the file header to work out the byte order, then goes through each captured packet. It handles common link types such as Ethernet, Linux SLL and BSD loopback before reading the IPv4 information. From there, it identifies TCP, UDP and ICMP traffic. For TCP packets, it also records the source and destination ports.

I also added some basic checks before the packet processing starts. If the wrong number of arguments is given, the script displays the correct usage and exits with status 1. It also checks that the supplied file actually exists and has a .pcap or .pcapng extension. I tested the missing-file case and the script correctly displayed Provide an existing .pcap or .pcapng file and returned exit status 2.

To test the script with an actual capture, I used three Ubuntu terminals. One ran a temporary HTTP server on port 8080, another used tcpdump to capture TCP traffic on the loopback interface, and the third sent two requests to the server using curl. After stopping tcpdump, I passed the resulting PCAP file to my script. It reported 8 packets, all of them TCP, with 16 endpoint references to 127.0.0.1. Port 8080 appeared 8 times, while the temporary client ports appeared twice each. The results matched the traffic I generated during the test.

Answers

1. Comparison with tcpdump/capinfos/Wireshark:

tcpdump reported 8 packets captured, and my Python script also counted 8 packets from the saved PCAP file. This gave me the same packet count as the capture tool.

2. Why both source and destination ports are counted:

TCP traffic has a client side and a server side, so recording both ports makes it possible to see which port belongs to the service and which ports are temporary client ports. In my capture, port 8080 was the server port, while the other ports were temporary client ports.

3. DNS counter:

I added a separate DNS counter that checks for TCP or UDP traffic involving port 53. This allows the script to show the number of DNS-related packets alongside the TCP, UDP and ICMP totals.

4. Limitation of port-based classification:

The main limitation is that a port does not always tell you what protocol or service is actually being used. For example, a service can run on an unusual port, while protocols such as DNS-over-HTTPS can use port 443. More accurate identification would require looking at the packet contents rather than relying only on port numbers.

Part 6 Integrated Security Challenge

Control	Command or evidence	Result	Conclusion
Address and route	Ip -br addr, ip route on both VMs	.101 and .102 On 192.168.56.0/24, no default gateway	Topology
Firewall policy	Sudo ufw status verbose	Deny incoming; SSH from .0/24; Telnet denied	Policy enforced
SSH key authentication	Journalctl -u ssh	Accepted publickey, matching fingerprint	Key auth working
Listening services	Ssh -tln	22,53,323 only	Minimal exposure
Host discovery	Authorised_ping_sweep .sh	3 hosts: .100, .101, .102	Matches inventory
PCAP summary	Pcap_summary.py	TCP port 22, endpoints .101/.102	Encryption confirmed
Cleanup	Pkill, ufw delete	No residual services or rules	Environment restored

Evidence

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu:~$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
enp0s3 UP 192.168.56.101/24 metric 1024 fe80::a00:27ff:fe7a:4925/64
santi@ubuntu:~$ ip route
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.101 metric 1024
santi@ubuntu:~$
```

```
(kali@kali)-[~]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.56.102/24 fe80::a00:27ff:fe5f:cc81/64
(kali@kali)-[~]
$ ip route
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.102 metric 100
(kali@kali)-[~]
$
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu:~$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
enp0s3 UP 192.168.56.101/24 metric 1024 fe80::a00:27ff:fe7a:4925/64
santi@ubuntu:~$ ip route
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.101 metric 1024
santi@ubuntu:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere # Allow SSH
22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
23/tcp DENY IN Anywhere # Block Telnet
8080/tcp ALLOW IN 192.168.56.102 # CORE102 test web
22/tcp (v6) ALLOW IN Anywhere (v6) # Allow SSH
23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

santi@ubuntu:~$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN Anywhere # Allow SSH
[ 2] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 3] 23/tcp DENY IN Anywhere # Block Telnet
[ 4] 8080/tcp ALLOW IN 192.168.56.102 # CORE102 test web
[ 5] 22/tcp (v6) ALLOW IN Anywhere (v6) # Allow SSH
[ 6] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

santi@ubuntu:~$
```

```
(kali@kali)-[~]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.56.102/24 fe80::a00:27ff:fe5f:cc81/64
(kali@kali)-[~]
$ ip route
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.102 metric 100
(kali@kali)-[~]
$
```

```
kali (Snapshot 3) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu:~$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
enp0s3 UP 192.168.56.101/24 metric 1024 fe80::a00:27ff:fe7a:4925/64
santi@ubuntu:~$ ip route
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.101 metric 1024
santi@ubuntu:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere # Allow SSH
22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
23/tcp DENY IN Anywhere # Block Telnet
8080/tcp ALLOW IN 192.168.56.102 # CORE102 test web
22/tcp (v6) ALLOW IN Anywhere (v6) # Allow SSH
23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

santi@ubuntu:~$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN Anywhere # Allow SSH
[ 2] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 3] 23/tcp DENY IN Anywhere # Block Telnet
[ 4] 8080/tcp ALLOW IN 192.168.56.102 # CORE102 test web
[ 5] 22/tcp (v6) ALLOW IN Anywhere (v6) # Allow SSH
[ 6] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

santi@ubuntu:~$
```

```
(kali@kali)-[~]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.56.102/24 fe80::a00:27ff:fe5f:cc81/64
(kali@kali)-[~]
$ ip route
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.102 metric 100
(kali@kali)-[~]
$ ssh -i ~/.ssh/core102_ed25519 santi@192.168.56.101
Enter passphrase for key '/home/kali/.ssh/core102_ed25519':
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-34-generic x86_64)

 * Documentation: https://docs.ubuntu.com
 * Management: https://landscape.canonical.com
 * Support: https://ubuntu.com/pro

System information as of Thu Oct 1 01:46:20 AM UTC 2026

System load: 0.02 Memory usage: 18% Processes: 145
Usage of /: 51.8% of 11.2GB Swap usage: 0% Users logged in: 1

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Oct 1 01:39:53 2026 from 192.168.56.102
santi@ubuntu:~$ whoami
santi
santi@ubuntu:~$ hostname
ubuntu
santi@ubuntu:~$ logout
Connection to 192.168.56.101 closed.
(kali@kali)-[~]
$
```

```
kali (Snapshot 4) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab5

Session Actions Edit View Help
santi@ubuntu:~$ mkdir -p ~/core102_lab5
cd ~/core102_lab5
santi@ubuntu:~/core102_lab5$ sudo mv ~/network_inventory.sh ~/core102_lab5/
cd ~/core102_lab5
chmod +x network_inventory.sh
bash -n network_inventory.sh
./network_inventory.sh
ls -l evidence/
[sudo: authenticate] Password:
Inventory saved to evidence/network_inventory_20261001_021020.txt
total 4
-rw-rw-r-- 1 santi santi 2505 Oct  1 02:10 network_inventory_20261001_021020.txt
santi@ubuntu:~/core102_lab5$

(kali@kali) [~/core102_lab5]
$ scp ~/core102_lab5/network_inventory.sh santi@192.168.56.101:~/
Enter passphrase for key '/home/kali/.ssh/id_ed25519':
network_inventory.sh 100% 667 567.8KB/s 00:00

(kali@kali) [~/core102_lab5]
$ cd ~/core102_lab5
./authorised_ping_sweep.sh 192.168.56
cat live_hosts*.txt
Authorised ICMP sweep of 192.168.56.0/24
Started: 2026-10-02T06:02:12-04:00
santi
```

```
kali (Snapshot 4) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab4

Session Actions Edit View Help
(kali@kali) [~/core102_lab5]
$ cd ~/core102_lab4
./service_check.py 192.168.56.101
CORE102 service check for 192.168.56.101
Started: 2026-10-02T06:05:28
TCP 22: OPEN
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

(kali@kali) [~/core102_lab4]
$
```

```
kali (Snapshot 4) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab5

Session Actions Edit View Help
santi@ubuntu:~$ mkdir -p ~/core102_lab5
cd ~/core102_lab5
santi@ubuntu:~/core102_lab5$ sudo mv ~/network_inventory.sh ~/core102_lab5/
cd ~/core102_lab5
chmod +x network_inventory.sh
bash -n network_inventory.sh
./network_inventory.sh
ls -l evidence/
[sudo: authenticate] Password:
Inventory saved to evidence/network_inventory_20261001_021020.txt
total 4
-rw-rw-r-- 1 santi santi 2505 Oct  1 02:10 network_inventory_20261001_021020.txt
santi@ubuntu:~/core102_lab5$

192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Finished: 2026-10-02T06:06:27-04:00
Total responsive hosts: 3
Saved: live_hosts_20261002_060212.txt
Authorised ICMP sweep of 192.168.56.0/24
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Authorised ICMP sweep of 192.168.56.0/24
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Authorised ICMP sweep of 192.168.56.0/24
Started: 2026-10-01T12:18:52-04:00
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Finished: 2026-10-01T12:23:06-04:00
Total responsive hosts: 3
Authorised ICMP sweep of 192.168.56.0/24
Started: 2026-10-02T05:54:19-04:00
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Finished: 2026-10-02T05:58:33-04:00
Total responsive hosts: 3
Authorised ICMP sweep of 192.168.56.0/24
Started: 2026-10-02T06:02:12-04:00
192.168.56.100 responded
192.168.56.101 responded
192.168.56.102 responded
Finished: 2026-10-02T06:06:27-04:00
Total responsive hosts: 3

(kali@kali)~[~/core102_lab5]
$ santi
santi: command not found

(kali@kali)~[~/core102_lab5]
```

```
kali (Snapshot 5) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

santi@ubuntu:~$ sudo tcpdump -i enp0s3 -w ~/core102_ssh.pcap 'tcp port 22 and host 192.168.56.102'
[sudo: authenticate] Password:
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C269 packets captured
270 packets received by filter
0 packets dropped by kernel
santi@ubuntu:~$ ls -lh ~/core102_ssh.pcap
python3 ~/pcap_summary.py ~/core102_ssh.pcap | tee ~/core102_ssh_summary.txt
-rw-r--r-- 1 tcpdump tcpdump 284K Oct  1 02:59 /home/santi/core102_ssh.pcap
File: /home/santi/core102_ssh.pcap
Packets: 5530
Protocols:
  TCP 269
Top endpoints:
  192.168.56.101 269
  192.168.56.102 269
Top TCP ports:
  22 269
  42908 221
  59402 48
santi@ubuntu:~$

(kali@kali)~[~/core102_lab4]
$ ssh -i ~/.ssh/core102_ed25519 santi@192.168.56.101 "whoami; hostname; exit"
Enter passphrase for key '/home/kali/.ssh/core102_ed25519':
santi
ubuntu

(kali@kali)~[~/core102_lab4]
$
```

Report

Operational Summary — CORE102 Lab 5 Integrated Challenge

By the end of the lab, the private network was working as expected on the 192.168.56.0/24 subnet. The Ubuntu server was using 192.168.56.101/24 on enp0s3, while the Kali client was using 192.168.56.102/24 on eth0. Neither machine had a default gateway, which was expected because this was a host-only network. The routing table on both systems showed the direct route for the lab network.

Verified controls

The Ubuntu server had UFW enabled with incoming connections denied by default and outgoing connections allowed. SSH was allowed from the lab subnet, while the temporary web service on port 8080 was restricted to the Kali client during testing. Telnet on port 23 was also explicitly blocked for both IPv4 and IPv6.

SSH key authentication was working from Kali using the dedicated Ed25519 key. The server logs showed a successful Accepted publickey login, and the SSH hardening configuration disabled password authentication. This confirmed that access was being controlled through the SSH key rather than a password.

I also checked the services listening on the Ubuntu server. SSH was listening on port 22, while the other services shown by `ss -tln` were restricted to the loopback interface. This meant SSH was the only service that could be reached across the lab network under the final configuration.

The authorised host-discovery script found three responsive addresses: 192.168.56.100, which was the VirtualBox host-only DHCP address, 192.168.56.101, the Ubuntu server, and 192.168.56.102, the Kali client. I ran the sweep three times and got the same results each time, including the timestamps and total count.

For the packet-capture test, I captured an authorised SSH connection between Kali and Ubuntu using `tcpdump`. The resulting PCAP showed TCP traffic between .101 and .102, with port 22 being the main service involved. I did not see usernames, passwords or commands in clear text because the SSH session was encrypted after the initial connection setup.

The service checker also behaved as expected. When I started the temporary Python web server on port 8080, the checker changed the result from CLOSED OR FILTERED to OPEN. After stopping the service, it changed back to CLOSED OR FILTERED. This showed that the checker was responding to the actual service state rather than displaying an old result.

Remaining issues

The main limitation was the lack of internet access on both VMs. Because of this, I could not install Scapy with pip. I worked around the problem by using a Python script based only on standard-library modules to read and summarise the PCAP. It was enough for the required TCP, UDP, ICMP and endpoint information, although it does not provide the deeper packet analysis that Scapy would have provided.

Another issue was the VirtualBox host-only adapter losing its IPv4 address after a reboot. When the host-only DHCP service was unavailable, the VMs had to be configured manually with their 192.168.56.x addresses before communication could be restored.

Cleanup

After completing the tests, I stopped the temporary Python web server and removed the temporary UFW rule for port 8080. I also removed the core102lab0 dummy interface created earlier in the labs and confirmed that it was no longer present.

The ~/core102_ssh.pcap capture was kept as evidence for the SSH traffic test. The dedicated SSH key pair at ~/.ssh/core102_ed25519 was also left in place until the instructor confirms that the evidence has been reviewed.

Part 7 Code Review Checklist

- ☑ Every script starts with the correct interpreter line (#!/usr/bin/env bash or #!/usr/bin/env python3).
- ☑ Bash scripts were checked with bash -n, and Python scripts were checked using py_compile.
- ☑ User input is checked before the scripts perform any network activity.
- ☑ The authorised network scope, 192.168.56.0/24, is clearly defined in the scripts and shown in their output.
- ☑ Network activity uses limits and timeouts, including ping -W 1 and a one-second socket timeout.
- ☑ Script output records timestamps and identifies the target being tested.
- ☑ Error messages clearly explain why a script stopped or rejected the input.
- ☑ No passwords, private keys, or authentication tokens are stored in any of the submitted files.

